



本小节内容

二叉树的前序中序后序遍历实战

二叉树的前序中序后序遍历实战

在上一小节我们实现了二叉树层次建树，因此我们在原有代码的基础之上，增加前序、中序、后序遍历代码，首先前序遍历是先打印自身，再打印左子树，再打印右子树，我们通过 `preOrder` 函数来实现，中序遍历是先打印左子树，再打印当前结点，再打印右子树，我们通过 `InOrder` 函数来实现，后序遍历是先打印左子树，再打印右子树，最后打印当前结点，我们通过 `PostOrder` 函数来实现，因为通过递归实现树的遍历实现简单，而且是考研常考的手法，因此我们均通过递归来实现树的遍历，下面直接上代码：

`function.h` 代码如下

```
#include <stdio.h>
#include <stdlib.h>

typedef char BiElemType;
typedef struct BiTNode{
    BiElemType c;//c 就是书籍上的 data
    struct BiTNode *lchild;
    struct BiTNode *rchild;
}BiTNode,*BiTree;

//tag 结构体是辅助队列使用的
typedef struct tag{
    BiTree p;//树的某一个结点的地址值
    struct tag *pNext;
}tag_t,*ptag_t;
```

下面是 `main.cpp` 的代码

```
#include "function.h"

//递归实现
//abdhiecfg 前序遍历，前序遍历就是深度优先遍历
void PreOrder(BiTree p)
{
    if(p!=NULL)
    {
```



```
        putchar(p->c); //等价于 visit 函数
        PreOrder(p->lchild);
        PreOrder(p->rchild);
    }
}
//中序遍历  hdbjefacg
void InOrder(BiTree p)
{
    if(p!=NULL)
    {
        InOrder(p->lchild);
        putchar(p->c);
        InOrder(p->rchild);
    }
}
//hdbjefacg 后序遍历
void PostOrder(BiTree p)
{
    if(p!=NULL)
    {
        PostOrder(p->lchild);
        PostOrder(p->rchild);
        putchar(p->c);
    }
}
//《王道 C 督学营》课程
//二叉树的建树（层次建树）
int main()
{
    BiTree pnew; //用来指向新申请的树结点
    char c;
    BiTree tree=NULL; //树根
    //phead 就是队列头，ptail 就是队列尾
    ptag_t phead=NULL,ptail=NULL,listpnew=NULL,pcur=NULL;
    //输入内容为 abcdefghij
    while(scanf("%c",&c))
    {
        if(c=='\n')
        {
            break;
        }
        pnew=(BiTree)calloc(1,sizeof(BiTNode)); //calloc 申请空间并对空间进行初始化，赋值为 0
        pnew->c=c; //数据放进去
        listpnew=(ptag_t)calloc(1,sizeof(tag_t)); //给队列结点申请空间
```



```
listpnew->p=pnew;
if(NULL==tree)
{
    tree=pnew;//树的根
    phead=listpnew;//队列头
    ptail=listpnew;//队列尾
    pcur=listpnew;
    continue;
}else{
    ptail->pnext=listpnew;//新结点放入链表，通过尾插法
    ptail=listpnew;//ptail 指向队列尾部
    //pcur 始终指向要插入的结点的位置
    if(NULL==pcur->p->lchild)//如何把新结点放入树
    {
        pcur->p->lchild=pnew;//把新结点放到要插入结点的左边
    }else if(NULL==pcur->p->rchild)
    {
        pcur->p->rchild=pnew;//把新结点放到要插入结点的右边
        pcur=pcur->pnext;//左右都放了结点后，pcur 指向队列的下一个
    }
}
printf("-----前序遍历-----\n");//也叫先序遍历，先打印当前结点，打印左孩子，打印右孩子
PreOrder(tree);
printf("\n-----中序遍历-----\n");//先打印左孩子，打印父亲，打印右孩子
InOrder(tree);
printf("\n-----后序遍历-----\n");//先打印左孩子，打印右孩子，最后打印父亲
PostOrder(tree);
return 0;
}
```